

Rapport de Projet – Memo-RPG

1. Introduction

Ce rapport présente le jeu Memo-RPG, projet de programmation en C réalisé dans le cadre du cours d'informatique. Memo-RPG est un jeu de mémoire compétitif multijoueur (2 à 4 joueurs) dans lequel des aventuriers parcourent un labyrinthe à la recherche de leur arme antique et d'un coffre au trésor, tout en affrontant les monstres qui s'y trouvent. Ce document détaille la composition de notre équipe, notre méthode de travail, l'architecture du programme, les fonctionnalités implémentées, les difficultés rencontrées et les solutions apportées, ainsi qu'un bilan de notre expérience.

2. Équipe et choix du projet

Notre groupe Mi1-K se compose de trois membres : Kamil Benslimane, Kyllian Turbak et Ambrine Abdouni. Nous nous sommes formés dès l'annonce du projet pour anticiper les délais et éviter les groupes formés à la dernière minute.

Parmi les six sujets proposés, nous avons choisi Memo-RPG pour plusieurs raisons. D'abord, le format ludique du projet nous a captivés : développer un véritable jeu plutôt qu'un outil de gestion nous semblait plus motivant. Ensuite, le sujet présentait un bon équilibre entre complexité technique et faisabilité, la gestion d'un plateau, de plusieurs joueurs et nous permettait de manipuler structures, énumérations et tableaux, sans pour autant exiger des concepts avancés non vu en cours. Ce choix a été validé à l'unanimité au sein du groupe.

3. Organisation et méthodologie

Pour faciliter notre collaboration, nous avons mis en place un dépôt Git hébergé sur GitHub, que chacun a cloné localement. Notre workflow consistait à effectuer un git pull avant chaque session de travail, puis à committer fréquemment avec des messages descriptifs (« Ajout du module combat », « Correction du bug d'affichage », etc.) et à pousser sur la branche principale. Nous avons choisi de travailler sur main plutôt que sur des branches séparées pour la simplicité, en se coordonnant via WhatsApp pour éviter les conflits sur les mêmes fichiers.

Le projet a été divisé en trois grands ensembles fonctionnels, un par membre. Kamil s'est chargé du module Plateau (création de la grille, mélange Fisher-Yates des 25 cartes, positionnement des joueurs sur les bords). Kyllian a pris en main le module Affichage et Sauvegarde (menus, affichage du plateau avec couleurs ANSI, chronomètre, persistance des statistiques dans save.txt). Ambrine s'est occupée du module Joueurs et Combats (création interactive des joueurs, choix d'armes à chaque tour, résolution des combats). Le module central de jeu (déroulement d'un tour, déplacement, traitement des événements) a été co-développé.

4. Architecture technique

Le code est organisé en onze modules, chacun ayant une responsabilité unique et définie. Ce découpage répond à plusieurs objectifs : faciliter la collaboration (chaque membre peut travailler sur son module sans interférer avec les autres), améliorer la lisibilité (aucun fichier ne dépasse quelques centaines de lignes), et permettre des corrections ciblées (un bug dans le combat se résout dans combat.c sans toucher au reste du code).

Voici les 11 modules :

- Cartes → Création des 25 cartes et mélange Fisher-Yates
- Plateau → Grille 5x5, positionnement des joueurs, reset/révélation
- Joueur → Structure Joueur, classes d'aventuriers, choix d'arme
- Combat → Résolution des combats arme/monstre, condition de victoire
- Jeu → Déroulement d'un tour, déplacement, switch sur événements
- Affichage → Plateau, bannières, couleurs ANSI, légendes
- Menu → Menu principal, fin de partie, choix des joueurs
- Saisie → Lecture sécurisée (entiers, chaînes, directions)
- Sauvegarde → Statistiques persistantes dans save.txt
- Chrono → Mesure du temps de partie
- Direction → Énumération NORD/EST/SUD/OUEST
- Main → Boucle principale du programme

Nous avons respecté plusieurs principes imposés par le cahier des charges : absence totale de variables globales (toutes les données circulent via des paramètres de fonction), définition des constantes via #define et utilisation d'énumérations typées plutôt que de constantes numériques pour les types de cartes, de monstres, d'armes et de directions.

5. Fonctionnalités implémentées

Dans notre projet Memo-RPG, nous avons réussi à implémenter plusieurs fonctionnalités importantes du cahier des charges. Le jeu se déroule sur une grille 5x5 mélangée aléatoirement grâce à l'algorithme de Fisher-Yates afin que chaque partie soit différente. Le plateau contient plusieurs types de cartes comme des monstres, des armes antiques, des coffres, un portail magique et des totems. Le jeu peut être joué de 2 à 4 joueurs, et chaque joueur choisit une classe d'aventurier parmi Guerrier, Ranger, Magicien ou Voleur. Chaque aventurier possède une arme antique spécifique à retrouver pendant la partie.

Nous avons aussi développé un système de combat où chaque arme active est efficace contre un seul type de monstre. Si le joueur possède la bonne arme, il gagne le combat et peut continuer son tour, sinon, il retourne à sa case de départ. Nous avons également ajouté des mécaniques spéciales comme le portail magique qui permet de se téléporter et le totem qui permet d'échanger des cartes du plateau. Une détection automatique du blocage a également été ajoutée : lorsqu'un joueur se retrouve entouré uniquement de cases déjà révélées, son tour se termine et il retourne à son point de départ. Une condition de victoire a été mise en place : le joueur doit retrouver son arme antique ainsi qu'un coffre pour gagner la partie.

Le plateau est affiché dans le terminal avec des couleurs ANSI afin de distinguer plus facilement les différents types de cases. Des menus interactifs permettent au joueur de naviguer dans le jeu et de choisir ses actions. Les déplacements peuvent être effectués à l'aide des touches Z, Q, S et D ou des directions numériques, ce qui rend la prise en main plus intuitive.

Les statistiques des joueurs sont sauvegardées dans un fichier afin de conserver le nombre de parties jouées et de victoires. Un tableau des scores trié permet également d'afficher les meilleurs joueurs ainsi que leurs pourcentages de victoire. De nombreuses vérifications ont été ajoutées afin de sécuriser les saisies utilisateur et limiter les risques d'erreurs ou de dépassements mémoire. L'ensemble de ces fonctionnalités a permis de développer un projet plus stable, plus complet et plus conforme aux exigences du cahier des charges.

6. Difficultés et solutions

Pendant le développement du projet, nous avons rencontré plusieurs difficultés techniques et d'organisations.

Difficultés 1 :

Au début du projet, le type `TypeArmeAntique` était défini dans deux fichiers différents (`joueur.h` et un autre fichier), ce qui empêchait la compilation et compliquait les comparaisons entre l'arme recherchée par le joueur et l'arme trouvée sur une case. Cela était dû à une mauvaise répartition, on a donc choisi de concentrer tout les énum dans `cartes.h`

Difficultés 2 :

Les premières versions utilisaient `scanf("%s", ...)` sans limitation de taille, ce qui exposait le programme à des dépassements de mémoire en cas de saisie trop longue. De plus, le caractère `\n` restait dans le buffer et polluait les saisies suivantes. On a alors créé un module `saisie.c` contenant trois fonctions sécurisées : `lire_entier(min, max)` avec validation des bornes, `lire_chaine(buffer, taille)` basée sur `fgets` avec retrait du `\n` via `strcspn`, et `lire_direction` acceptant lettres ZQSD ou chiffres.

Difficultés 3 :

Quand un joueur effectuait plusieurs sous-tours consécutifs (combat gagné, puis coffre, etc.), plusieurs plateaux s'empilaient à l'écran sans effacement, c'était parce que l'effacement de l'écran ne se faisait qu'au début du tour, pas entre les sous-tours. On a donc refactorisé `tour_joueur` pour systématiquement effacer l'écran avant chaque sous-tour et après chaque déplacement. La fonction prend désormais en paramètre l'index du joueur et l'instant de début pour pouvoir réafficher la bannière.

Difficultés 4 :

Les premiers menus bouclaient à l'infini car la fonction `menu_principal` était déclarée `void` et ne demandait jamais de saisie à l'utilisateur, c'était dû à une confusion entre déclaration et utilisation des valeurs de retour. On a changé la signature pour retourner un `int` (le choix de l'utilisateur), appel de `lire_entier(1, 3)` dans la fonction, et utilisation du retour dans `main.c` pour orienter la boucle principale.

7. Bilan et apprentissages

Ce projet nous a permis d'approfondir notre maîtrise du langage C au-delà de ce que les TD nous avaient apporté. La manipulation des structures, le passage par pointeurs, la gestion des saisies et l'utilisation de modules ont été de bonnes compétences à développer. Nous avons également découvert Github, et appris à coordonner notre travail à plusieurs sur un même projet.

Le travail d'équipe a été enrichissant et exigeant : synchroniser les commits, résoudre les conflits de merge, et maintenir une cohérence sur l'ensemble du code ont demandé une discipline que nous n'avions pas anticipée. La communication régulière via WhatsApp s'est révélée essentielle pour éviter de se marcher sur les pieds.

Avec davantage de temps, nous aurions aimé approfondir certaines pistes : un affichage du chronomètre en temps réel, un mode joueur contrôlé par IA, ou encore une interface graphique. Cela resteraient à explorer dans un projet ultérieur.

8. Conclusion

Ce projet Memo-RPG nous a permis de réaliser un jeu complet en langage C, respectant l'intégralité des fonctionnalités demandées : grille mélangée aléatoirement, gestion de 2 à 4 joueurs, mécaniques de combat, portail, totem, continuation du tour, détection du blocage, sauvegarde persistante des statistiques et chronomètre. Au-delà des exigences, nous avons enrichi le projet avec une interface soignée. Ce projet nous a fait grandir techniquement et humainement : nous avons appris à découper proprement un programme, à sécuriser nos saisies, à coordonner un travail à plusieurs et à respecter les délais. Ce sont des compétences sur lesquelles nous pourrions capitaliser dans nos futurs projets d'ingénieur. Le dépôt complet du projet est accessible sur GitHub à l'adresse [KyKyto-Cy/Projet-Mi-1-MemoRPG](https://github.com/KyKyto-Cy/Projet-Mi-1-MemoRPG)